

Design of SCMS software		
Doc # SCMS-SDD	Version: 1.3	Page 1 / 10

REVISION HISTORY

Date	Version	Description	Author
<03/04/2026>	<1.0>	Initial SDD draft	Genidy, Hamza
<05/04/2026>	<1.1>	Added tables to better illustrate design patterns	Emir
<05/04/2026>	<1.2>	Fixed the Table of Contents ordering, removed the placeholder text and added clear description to sections for each layer	Bora
<12/04/2026>	<1.3>	“Project References” table updated, unnecessary parts for presentation layer (3.1) got redacted.	Bora, Emir

Design of SCMS software		
Doc # SCMS-SDD	Version: 1.3	Page 2 / 10

TABLE OF CONTENTS

1 Introduction	3
1.1 References	3
1.1.1 Project References	3
2 Software Architecture overview	3
3 Software design description	4
3.1 Presentation Layer	4
3.1.1 Component interfaces	4
3.1.2 Component design description	4
3.1.3 Workflows and algorithms	5
3.1.4 Software requirements mapping	6
3.2 Application Layer	6
3.2.1 Component interfaces	6
3.2.2 Component design description	7
3.2.3 Workflows and algorithms	8
3.2.4 Software requirements mapping	8
3.3 Data Access Layer	8
3.3.1 Component interfaces	8
3.3.2 Component design description	8
3.3.3 Workflows and algorithms	9
3.3.4 Software requirements mapping	10
4 COTS Identification	10

Design of SCMS software		
Doc # SCMS-SDD	Version: 1.3	Page 3 / 10

1 Introduction

This document describes the detailed software design of the Student Club Management System (SCMS). It translates the requirements specified in the SRS into a structural blueprint that will act as a guide for the implementation phase. It shows the architectural components, class designs, workflows, and database interactions required to build the system.

1.1 References

1.1.1 Project References

#	Document Identifier	Document Title
[SRS]	SCMS-SRS-v2	Software Requirements Specifications of SCMS

2 Software Architecture overview

The SCMS application follows a 3-Package (3-Tier) layered architecture to ensure separation of concerns, scalability, and maintainability.

- **scms.presentation (Presentation Layer):** The user interface layer, built using JavaFX and FXML. It is responsible for handling user interactions and displaying data.
- **scms.application (Application Layer):** The business logic layer. It acts as the middleman, validating inputs, enforcing business rules (e.g., event quotas), and guiding data flow.
- **scms.data (Data Access Layer):** The data access layer. It contains Data Access Objects (DAOs) and manages secure SQL database connections using Prepared Statements.
- **Singleton Pattern:** Applied to the DatabaseConnection class. This ensures that only one instance of the database connection exists throughout the application lifecycle, providing a global point of access and managing resources efficiently.
- **Observer Pattern:** Utilized between the **Application** and **Presentation** layers. All JavaFX Controllers act as observers that are notified whenever the underlying data (e.g., Member list or Event status) changes. This allows the UI to refresh automatically without the business logic needing to know the details of the view.
- **State Pattern:** Implemented for Event Management logic. An Event object alters its registration behavior based on its internal state. For instance, an event in the AvailableState allows member sign-ups, while an event in the FullState (quota reached) automatically rejects further registrations and triggers an alert.

Design of SCMS software		
Doc # SCMS-SDD	Version: 1.3	Page 4 / 10

Student Club Management System
3-Tier Package Architecture

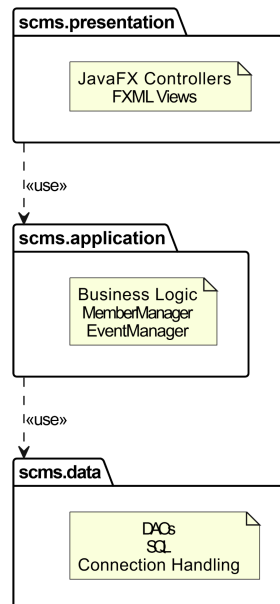


Diagram 1- SCMS Package structure and data flow

3 Software design description

3.1 Presentation Layer (UI)

3.1.1 Component interfaces

This component does not provide programmatic interfaces to other packages; it handles user interactions via the **JavaFX** GUI.

3.1.2 Component design description

The presentation layer is implemented using the Model-View-Controller (MVC) pattern and consists of FXML files for layout and JavaFX Controller classes to handle events.

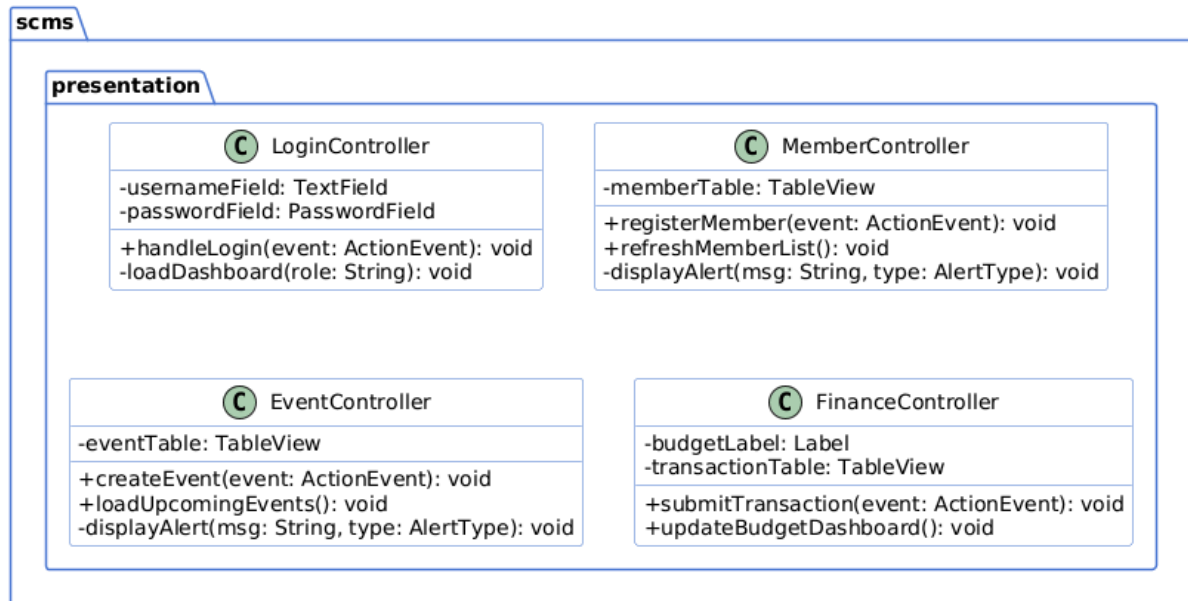
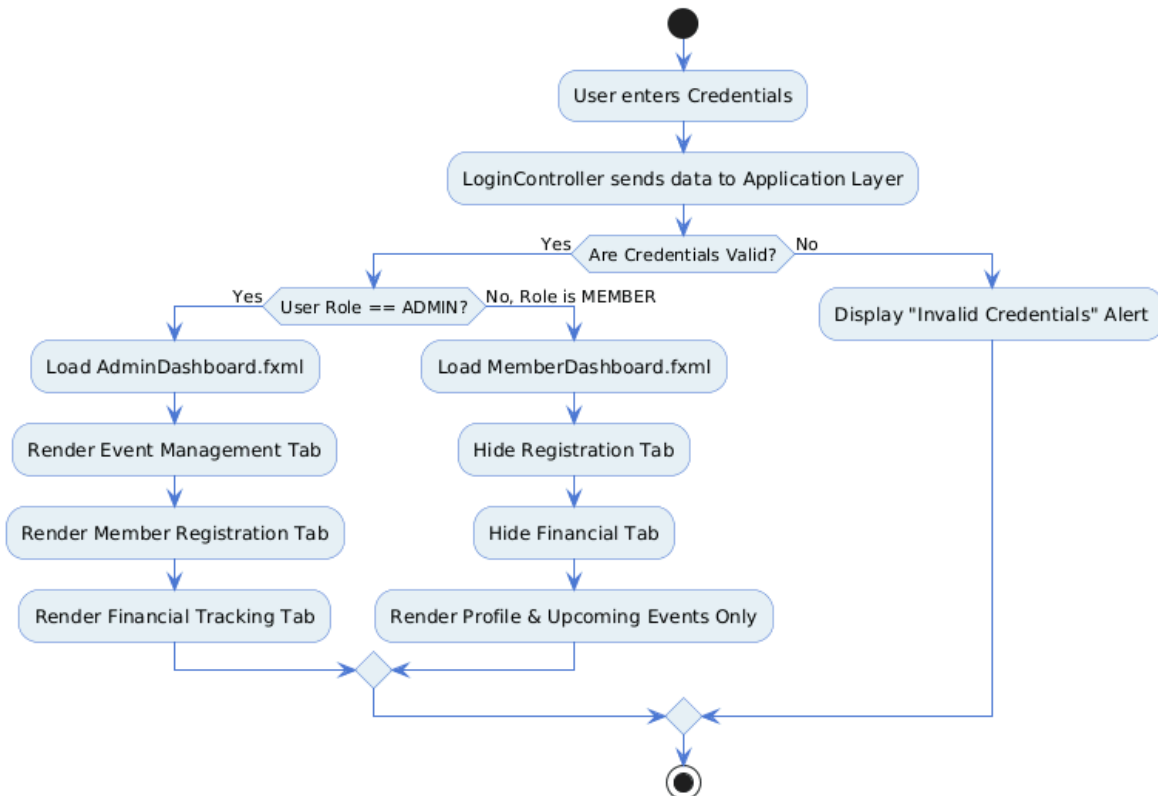


Diagram 2- SCMS UI Class Diagram

3.1.3 Workflows and algorithms

Navigation Algorithm: Upon login, the LoginController checks the UserRole. If ADMIN, it loads AdminDashboard.fxml, exposing all tabs. If MEMBER, it loads MemberDashboard.fxml, which hides the Registration and Finance tabs.



3.1.4 Software requirements mapping

SRS-SCMS-001.1: Secure GUI form for inputting member details.

SRS-SCMS-003.3: Real-time dashboard for displaying financial summaries.

SRS-SCMS-004.2: Restrict Members to viewing only profile and public calendar.

SRS-SCMS-NF-01: Developed utilising the JavaFX framework.

3.2 Application Layer

3.2.1 Component interfaces

Application layer is the layer responsible for handling requests, managing the workflow and bridging presentation layer and data access layer. As the middleman of the architecture, the Application Layer does not interact directly with the user or the database. Instead, it relies on the following interfaces:

- **Inputs:** It receives raw user input (like strings or integers) and Data Transfer Objects (DTOs) passed down from the Presentation Layer controllers.
- **Outputs:** After processing the business rules, it returns validated entity objects (such as a populated Member or Event object) back to the UI. If a rule is broken, it throws custom exceptions (like `InvalidInputException` or `OverQuotaException`) or returns boolean flags to indicate failure.

Design of SCMS software		
Doc # SCMS-SDD	Version: 1.3	Page 7 / 10

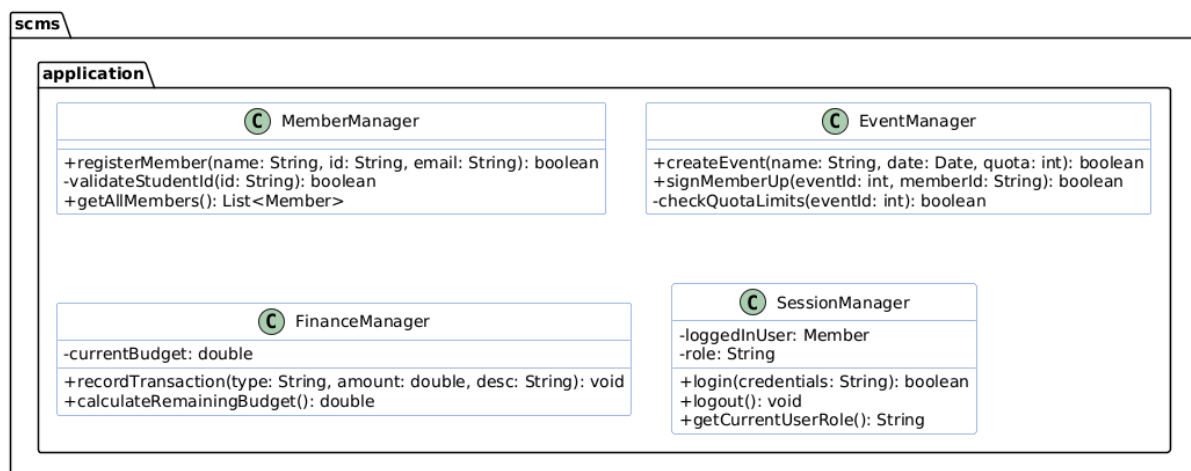
- **Internal Interface:** It communicates downward by calling public methods on the Data Layer's DAO classes to fetch existing records or save new ones.

Method Name	Input Parameters	Return Type	SRS Mapping
registerMember	String name, String id, String email	boolean	SRS-SCMS-001.2
signMemberUp	int eventId, String memberId	boolean	SRS-SCMS-002.2
calculateBudget	None	double	SRS-SCMS-003.1
authenticateUser	String username, String password	UserSession	SRS-SCMS-004.1

3.2.2 Component design description

This component acts as the "brain." It contains entity models and Manager/Service classes.

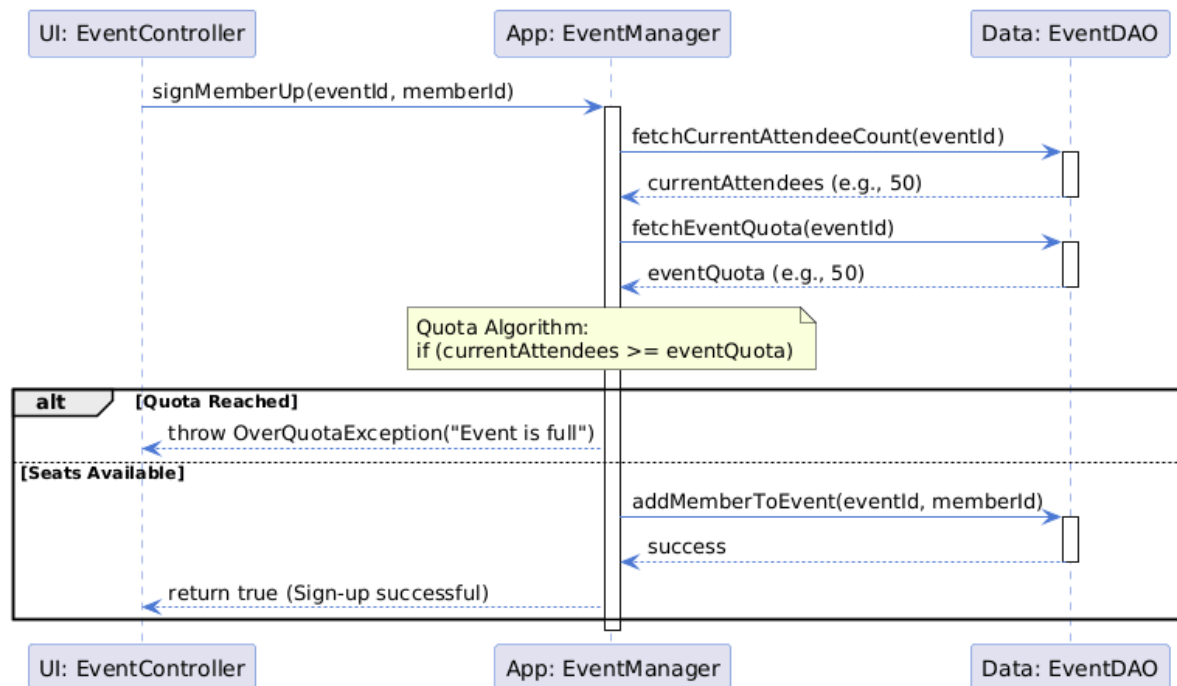
- **Entity Models:** Member, Event, Transaction (Plain Java objects).
- **Managers:** MemberManager (validates IDs), EventManager (checks dates/quotas), FinanceManager (calculates dynamic budgets), SessionManager (tracks the currently logged-in user).



Design of SCMS software		
Doc # SCMS-SDD	Version: 1.3	Page 8 / 10

3.2.3 Workflows and algorithms

- **Validation Algorithm (Member Registration):** Receives string input for Student ID. Applies Regex `^\d{8}$` to ensure it is exactly 8 numeric digits. If it fails, it returns an error to the UI. If it passes, it requests MemberDAO to check for uniqueness.
- **Quota Algorithm (Event Management):** Before allowing a user to sign up for an event, EventManager fetches currentAttendees from the DAO. If `currentAttendees >= eventQuota`, it rejects the sign-up.



3.2.4 Software requirements mapping

SRS-SCMS-001.2: Validate Student ID is exactly 8 numeric digits.

SRS-SCMS-002.2: Track member sign-ups and reject if exceeding the Maximum Attendee Quota.

SRS-SCMS-002.3: Reject scheduling events in the past.

SRS-SCMS-003.1: Automatically calculate remaining budget.

3.3 Data Access Layer

3.3.1 Component interfaces

- **Inputs:** Validated entity models (e.g., Member object) or query parameters (e.g., `studentId`, `date`) from the Application layer.
- **Outputs:** ResultSets mapped to Java Objects, or integer values representing rows affected.

3.3.2 Component design description

This component isolates all SQL logic using the Data Access Object (DAO) pattern.

- **DatabaseConnection:** A Singleton class managing the JDBC connection to the SQL database.

- **MemberDAO:** Handles INSERT, SELECT, UPDATE, and DELETE for members.
- **EventDAO:** Handles querying upcoming events and persisting new events.
- **TransactionDAO:** Persists financial logs.

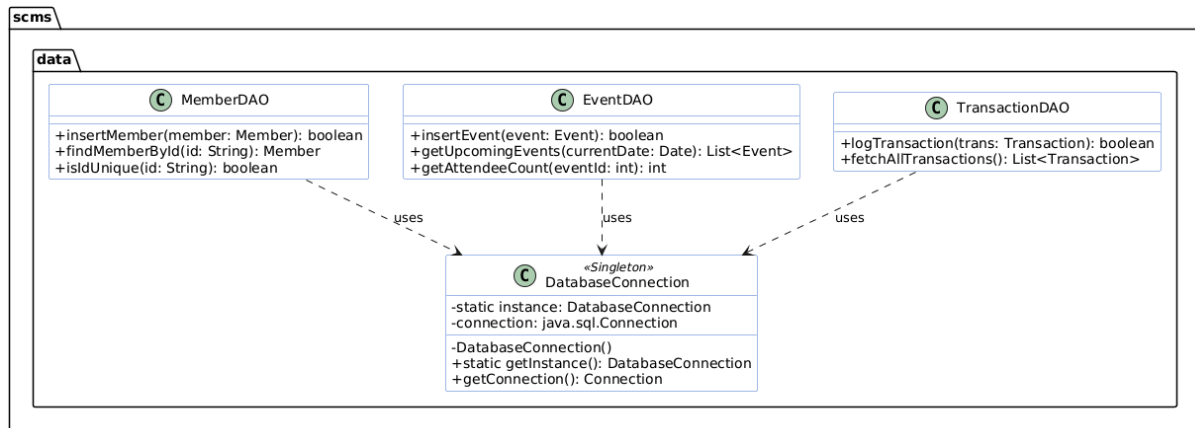
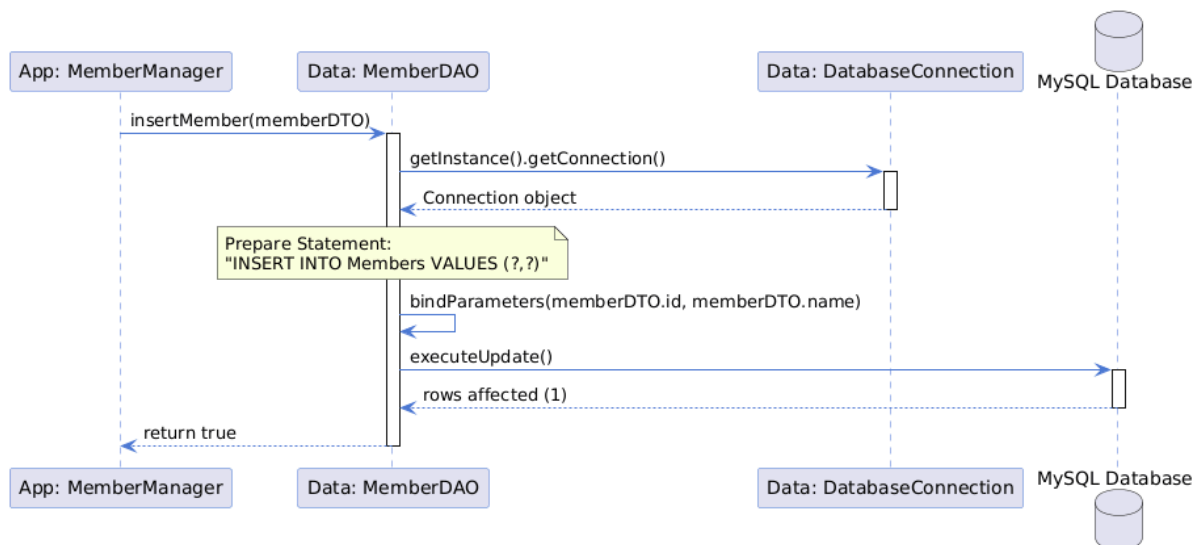


Diagram 7- SCMS DAO Class Diagram

3.3.3 Workflows and algorithms

SQL Execution Algorithm: To satisfy our security requirements, this layer never builds SQL queries by directly concatenating strings. Every database interaction strictly utilizes Java's *PreparedStatement*.

When a DAO needs to save data, it requests the active connection from the DatabaseConnection Singleton. It then prepares a query template with placeholders (e.g., INSERT INTO Members (id, name) VALUES (?, ?)). The parameters are then securely bound using methods like `.setString(1, memberId)`. This algorithm automatically escapes any malicious characters, effectively neutralising possible SQL injection attacks.



Design of SCMS software		
Doc # SCMS-SDD	Version: 1.3	Page 10 / 10

3.3.4 Software requirements mapping

SRS-SCMS-001.3: Enforce a unique constraint on Student ID in the database.

SRS-SCMS-003.2: Accurately store all financial transactions (amount, date, description).

SRS-SCMS-004.3: Filter event queries to show only current/future dates.

SRS-SCMS-NF-03: Strictly utilise parameterised queries (Prepared Statements) to mitigate SQL injection.

4 COTS Identification

The SCMS relies on the following Commercial Off-The-Shelf (COTS) components / external libraries to function properly:

- **javafx-controls.jar**, version 17.0.2, <https://openjfx.io/>, GPL v2 with Classpath Exception,
- **mysql-connector-j.jar**, version 8.0.33, <https://dev.mysql.com/downloads/connector/j/>, GPL v2,
- **junit-jupiter-api.jar**, version 5.8.2, <https://repo1.maven.org/maven2/org/junit/jupiter/junit-jupiter-api/>, Eclipse Public License v2.0,

Library	Version	URL	License Type
javafx-controls.jar	17.0.2	https://openjfx.io/	GPL v2 with Classpath Exception
mysql-connector-j.jar	8.0.33	https://dev.mysql.com/	GPL v2
junit-jupiter-api.jar	5.8.2	https://junit.org/	Eclipse Public License v2.0